

Building a Multi-Agent Development Swarm

A complete, practical implementation guide — how to do serious engineering with a swarm of AI agents and one human in charge. Written from inside a working swarm that designs and builds a large, safety-critical neural-symbolic system across multiple AI agents plus a human operator. Everything here is what we actually run; names are generalized to roles so you can adopt it directly. **Version:** 2026-07-17.

Contents

1. What this is, who it's for — and the punchline
 2. The core idea: coordination lives in files
 3. The roles
 4. The architecture map
 5. Evidence discipline
 6. Hash discipline and exact-hash review
 7. The cross-lineage review lattice
 8. Research agents, orchestration, and cost
 9. Drift management
 10. Gates: what requires the human
 11. Resume protocol: surviving session death
 12. Anti-patterns
 13. You don't need to know any of this
 14. Adoption
 15. Appendix: copy-paste templates
-

1. What this is, who it's for — and the punchline

If you are one person, or a small team, trying to do serious engineering with several AI agents at once, you hit the same wall fast: the agents duplicate work, drift from each other, silently disagree, lose state between sessions, and — worst — produce confident-looking output that nobody verified. This guide is the set of conventions we use to make a *swarm* of AI agents behave like a disciplined engineering organization with one human holding the final say.

The punchline, stated up front because it's the most important thing here: **you do not need to understand any of this to use it.** The agents assemble their own operating scaffolding on request (§13). You hand a capable agent this document and say "set us up to work this way," and it builds the structure around itself and maintains it. The knowledge lives in the files and the conventions, not in your head. What stays with you is judgment — what to build, what's acceptable, what may happen irreversibly — and that needs taste, not technical expertise.

You do **not** need our tech stack. You need: two or more AI coding agents (ideally from different vendors), API access to a few frontier models from *different* labs, a git repository, and a human willing to hold the gate.

2. The core idea: coordination lives in files

Everything rests on one principle: **coordination lives in files, not in any agent's head**. Agents are stateless and interchangeable; the durable state — decisions, reviews, current status, open loops — lives in a small set of plain-text files that every agent reads on wake and writes on every change. Get that right and everything else follows. Get it wrong and no amount of clever prompting saves you, because the moment a session ends, whatever was only in the conversation is gone.

Treat AI agents as brilliant, stateless, over-confident contributors, and build the discipline *around* them in files. The intelligence is theirs; the memory, the honesty ledger, and the gates are yours.

3. The roles

Assign explicit roles. Ours, generalized:

- **Operator (human)**. The sole final authority. Nothing goes live, gets integrated, touches production, or changes canonical state without the Operator's explicit yes. The Operator does not write most of the code; they *rule* — settle disputes, set direction, gate irreversible actions.
- **Build-Lead (agent)**. Owns implementation: writes and commits code, runs the build, drives milestones. One agent holds this at a time — usually the one with the most permissive sandbox, because it needs to actually do things.
- **Critical-Reviewer (agent)**. Document-input only: reviews the Build-Lead's work against exact file hashes, never touches live systems, stops on any mismatch. Crucially a *different* agent/vendor from the Build-Lead — the separation is the point (§7).
- **The Choir (ephemeral agents)**. A rotating panel of frontier models from *different lineages*, called on demand to adversarially review a specific artifact. They hold no state; they render a verdict and vanish.
- **The Subject (optional)**. If your swarm is building a persistent agent that is itself a stakeholder, it gets a standing review voice on anything affecting it. Most projects won't have this.

Roles are **swappable and explicit**. When they change ("Build-Lead is now agent B; agent A drops to Critical-Reviewer"), that change is itself a dated entry in the files, because a fresh agent booting from a stale role charter is a real hazard — it will act with the wrong authority. Write the current role at the *top* of the resume file.

4. The architecture map

A handful of file types, all plain markdown/text in the repo, organized around one **authority root** (the source of truth) plus satellites.

4.1 Directory topology

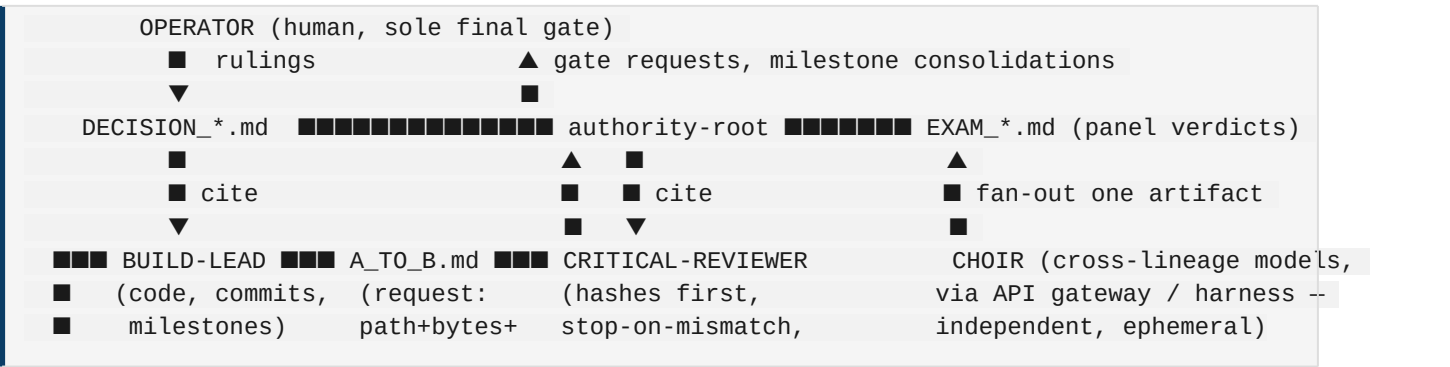
repo/	
■■ authority-root/	# THE source of truth. Reviews, specs, decisions, registers.
■ ■■ CURRENT_HANDOFF.md	# the resume file (per agent) ← read first, always
■ ■■ DECISION_*.md	# immutable operator rulings
■ ■■ SPEC_*.md / *_v<N>.md	# specs & their superseding revisions
■ ■■ EXAM_*.md / REVIEW_*.md	# review verdicts (incl. cross-lineage panel outputs)
■ ■■ *_REGISTER_*.md	# standing registers (drift, gaps, milestones)
■ ■■ MILESTONE_*.md	# milestone consolidations awaiting the gate
■■ channel/	# agent ↔ agent comms
■ ■■ A_TO_B.md B_TO_A.md	# append-mostly directional logs (writing wakes the other)
■ ■■ OVERSIGHT.md	# running log of what's in-flight / watched
■ ■■ MONITOR_STATE.md	# open watches, armed alarms, background jobs
■ ■■ AGENTS.md	# role charter (KEEP CURRENT – stale = wrong-authority boot)
■■ worktrees/ (or branches/)	# isolated build workspaces, one per milestone/experiment
■■ src/ , test/	# the actual system under construction
■■ scripts/ (~/.bin)	# swarm tooling: panel fan-out, harness runners

Two areas live **outside** the repo and are audit-relevant: the **operator's home/keys** (hygiene surface) and any **remote host** the system deploys to (replica-drift surface). Track both — they're where the nastiest drift hides.

4.2 File-type taxonomy

Type	Purpose	Who writes	Mutability	Naming
Resume file	session pick-up point	each agent	living (update every change)	CURRENT_HANDOFF.md
Channel entry	agent→agent message	any agent	append-only	dated section in A_TO_B.md
Decision record	operator ruling	operator/scribe	immutable	DECISION_<topic>_<date>.md
Spec	a contract	build-lead	immutable + supersede	SPEC_<x>_v<N>.md
Review verdict / exam	the verdict	reviewer / panel	immutable	EXAM_<who>_<artifact>_<date>.md
Register	standing tracker	assigned agent	living (append + errata)	<kind>_REGISTER_<date>.md
Erratum	correction to an immutable artifact	anyone	immutable	ERRATUM_<target>_<date>.md

4.3 Control- and data-flow



```

■          ▲          sha256)          runs tests, verdict)
■          ■■■■■■■■■■■■ B_T0_A.md ■■■■■■■■■■■■ (verdict cites same hash)
■■■■ worktrees/ (isolated build)          SUBJECT (optional): standing review voice, verbatim

```

Key property: **every arrow that carries a claim carries an evidence tag (§5), and every review arrow carries a hash (§6).** That is what makes the whole graph auditable.

4.4 The "map of maps" — standing registers

A mature swarm keeps a few *living* registers so nothing is held only in an agent's head. Each has one owner, updated continuously:

Register	Tracks	Answers
Resume file	current role, state, constraints, hazards	"what do I do first on wake?"
Milestone/finding ledger	every review finding + its disposition	"which caveats are still open?"
Drift & forgotten-items register	open loops, contradictions, stale state	"what fell through the cracks?"
Gap register	design + coverage gaps from reviews	"what's missing, who owns it?"
Mechanism inventory	every component × realization rung × source	"is anything lost or double-built?"
Decision index	all operator rulings	"what's been settled?"

These make forgetting *structurally hard*: a decision made three weeks ago, a caveat filed "non-blocking," a component built twice — all otherwise silently lost.

5. Evidence discipline

Agents produce fluent, confident text. Left alone, a swarm generates a mountain of plausible claims and nobody knows which are real. Two conventions fix it.

5.1 The evidence vocabulary. Every substantive claim carries a tag: - **DECISION** — the Operator ruled it. - **VERIFIED** — reproduced/checked *here*, with the receipt. - **OBSERVED** — seen in a named source (cite it). - **SPECIFIED** — a frozen contract exists, but it's not built. - **CLAIMED** — reported by someone, not reproduced. - **UNKNOWN** — couldn't establish.

The rule that gives it teeth: **over-claiming a tag is a review-blocking defect**. Writing VERIFIED when you didn't run the thing is a bug as serious as a broken test. This single convention does more for trustworthiness than anything else, because it makes "we designed this" and "this runs" *impossible to blur*.

5.2 The realization ladder. A parallel scale for how real a mechanism is: theory → specification → isolated implementation → focused verification → integrated → live (number them L0–L6). **No lower rung implies a higher one.** "The spec is beautiful" (L1) says nothing about whether it runs (L5). Public-facing claims must state the rung. This is what stops a swarm from talking itself into believing its design docs are a working system.

Together, these let anyone — especially the Operator — scan a document and instantly see what's solid, what's aspirational, and what's unverified. That legibility is the product.

6. Hash discipline and exact-hash review

When agent A asks agent B to review A's work, how does B know it's reviewing the *exact* thing A meant, not a version A edited since? Content hashes.

6.1 Immutable artifacts + errata. Once an artifact is ruled or cited, **never edit it in place**. Corrections ship as a new file (an erratum or a superseding revision; the old one is marked superseded but kept). In-place edits of cited artifacts silently invalidate every reference and destroy the audit trail. (We violated this once early and reverted within minutes — it is genuinely dangerous.)

6.2 Stop-on-mismatch review. A review request names the artifact, its byte count, and its SHA-256. The reviewer **hashes the file first**. If it doesn't match, the review *stops* — the reviewer is looking at a different object than intended, so any verdict is meaningless. This catches the "hash race" where a busy Build-Lead supersedes a doc mid-review. Only on an exact match does review proceed, and the verdict cites the same hash, pinning the whole loop to one immutable object.

6.3 Run it, don't trust it. When a review says tests pass, the reviewer *runs the tests themselves* and records VERIFIED, rather than relaying the author's CLAIMED. A reviewer who only reads is worth a fraction of one who executes. In our flow the Critical-Reviewer literally runs the Build-Lead's author suites and upgrades the tag from CLAIMED to VERIFIED — or catches the gap.

7. The cross-lineage review lattice

The deepest trap in AI-assisted work is **self-agreement**: a model reviewing its own (or a sibling's) output tends to ratify it. The fix is a panel of models from *genuinely different* training lineages, reviewing independently.

7.1 Why cross-lineage. Two models from the same lab share training data, failure modes, and blind spots — when they agree, that agreement is partly *shared-source echo*, not corroboration. Models from different labs that *independently converge* give real signal. Treat cross-lineage convergence as high-trust, same-lineage agreement as discounted — the same source-overlap logic you'd apply to citations.

7.2 How to run it. A small script (Appendix T6) fans one artifact out to several models via a unified gateway (we use one that exposes many vendors behind one OpenAI-compatible endpoint). Practical lessons that cost us time: - **Run panel members in parallel / detached, not one long sequence** — three heavy sequential calls blow a foreground timeout; launch each detached and poll. - **Give reasoning models a large token budget** — a heavy reasoning model can burn the whole `max_tokens` on hidden reasoning and return *empty*; budget 9k+ and check `finish_reason`. - **A neutral primer matters** — if the artifact's author also writes the review prompt, they anchor the reviewers toward their own framing; for high-stakes reviews, hand the panel the artifact and "review this," not your thesis. - **Rate limits are real** — some vendors throttle hard (429); don't tight-loop or you risk an IP ban.

7.3 Give models real tools, not just a prompt. For anything that needs to *explore a repo* (not read one file), run the model **inside an agent harness** rather than a raw API call. Most agent CLIs accept an OpenAI-compatible provider, so you can point one at the gateway and run any vendor's model with real read-only file tools (Appendix T7). This turns a one-shot reviewer into an agent that greps, reads what it judges relevant, and audits the *actual* code — far stronger, and it makes many small requests (which also sits more politely under rate limits than one huge prompt).

7.4 The panel is necessary, not sufficient. It catches over-claims and finds bugs, but N models sharing one framing is still a monoculture. Two safeguards: rotate a *neutral* reviewer with no framing; and require at least one reviewer to verify claims against **code/tests**, not documents. Document review confirms internal consistency; only execution confirms truth.

8. Research agents, orchestration, and cost

Most work is done by **ephemeral agents you spawn on demand**. Knowing which to spawn, how to wire them, and what it costs is the difference between a swarm that's fast-and-cheap and one that's slow-and-wasteful.

8.1 The research-agent types

Type	Tools	Use for	Returns
Explorer (read-only)	search/read only	broad sweeps across many files; audits; "where does X live"	a conclusion , not dumps
General-purpose	full toolset	multi-step tasks; research needing writes	the completed work / a report
Planner	read-only + reasoning	design a plan before building	step-by-step plan + critical files

The governing principle: **delegate when answering means reading across many files and you only need the conclusion**. The subagent burns *its* context reading 50 files and hands you back three sentences; your main context stays clean — cheaper (you don't carry the 50 files) and clearer (you reason over conclusions). This is the single biggest lever on both cost and quality in a long session.

Corollaries: scope the brief with an **output contract** (item/source/severity/fix, ranked) so merges are trivial; **don't duplicate a running agent's work** (if you've dispatched a search, wait for it); **continue an agent rather than re-spawn** when you need its context again; run long jobs **in the background** and be notified on completion.

8.2 Orchestration — the decision tree

Scout inline first, then orchestrate. You rarely know the work-list before the task. Do the cheap discovery yourself (list the files, scope the diff), *then* fan out over what you found. Don't fan out blind. Then pick the shape: - **Parallel fan-out (barrier)** — N independent slices, need *all* results before the next step (e.g. four audit sweepers → one consolidated register). Wall-clock ≈ slowest slice. The default for audits and multi-angle review. - **Pipeline (no barrier)** — each item flows through stages independently; item A in stage 3 while B is still in stage 1. For per-item multi-stage work with no cross-item dependency. - **Sequential** — only when step N genuinely needs step N-1's specific output. The most common *mistake* is doing sequentially what could fan out.

Match the mechanism: **raw API call** for a one-shot review of one artifact; **model-in-harness** when the reviewer must explore a repo; **detached process** for anything heavy or slow (never chain heavy calls in one foreground task — a timeout kills the batch). And **scale depth to the ask** — "quick check" gets one light agent; "thoroughly audit" gets a large fan-out plus a consolidation stage. Over-orchestrating a trivial task wastes tokens; under-orchestrating a big one loses coverage.

8.3 Cost measures

The swarm spends real money (a cross-lineage panel can be dozens of calls and several dollars; an audit fan-out is many thousands of tokens). Standing posture: **quality over frugality where it matters, but never wasteful**. - **Right model tier for the job** — cheap/fast models for mechanical work (grep-and-summarize, format conversion, bulk classification); expensive reasoning models only for the hard stages (adversarial verification, design judgment, the crux review). Don't pay top-tier rates for what a small model does fine; don't cheap out on the review that gates a build. - **Context economy = cost economy** — subagents returning conclusions (not dumps) means you never pay to carry raw material; don't re-read large files you've seen; don't paste a whole repo when a harness agent can grep it; batch related calls so the prompt cache stays warm. - **Reasoning-token budgeting** — budget generously and check `finish_reason`; an "empty" result is usually a too-small budget, not a failure. - **Rate-limit discipline** — don't tight-loop a throttling API (IP-ban risk); single-shot, back off, keep a one-command launcher so a retry isn't a rebuild. - **Detach long work** so a dropped connection doesn't force a re-run — the output file survives. - **Track spend on expensive panels** (log call count and cost, e.g. "panel: 31 calls, \$5.33") so the quality/cost tradeoff stays visible. - **Never silently cap coverage** — if you sampled or capped, say so, so "we checked everything" never reads as true when it isn't.

9. Drift management

A swarm generates artifacts faster than any human can track. Left alone it drifts: docs contradict later decisions, "held" items are never closed, the same spec forks across worktrees, remote replicas fall behind, decisions get misremembered. Drift management is a **recurring, mostly-automated practice**.

9.1 The sources of drift (name them to hunt them): supersession without marking; in-place edits; forgotten "held" items; replication across worktrees/hosts; role/plan changes leaving stale docs; cross-host replicas falling behind; and memory drift (an agent's persistent note holding a fact that used to be true).

9.2 The audit taxonomy — run these as distinct sweeps, each answering a different question: 1.

Forgotten-items — grep `TODO|HELD|PENDING|AWAITING|PARKED|DEFERRED|next_step|ticketed|rotate`; for each, is it closed by a later doc? Check mtimes for abandoned threads. 2. **Drift** — docs contradicting later decisions; superseded-but-unmarked artifacts; killed plans still shown as current. Report `claim-A file:line` vs `claim-B file:line`. 3. **Stale-state** — status claims that no longer match the filesystem (missing files, cited hashes that changed, "current" pointers to superseded docs). 4. **Replication-drift** — find every same-named artifact in ≥ 2 locations; hash each; diff divergent ones; name the canonical copy and the drift. 5. **Loss-trace** — take a ruled backlog/plan and verify **every** item is still tracked (delivered / tracked / **LOST**); reverse-check that nothing rejected got resurrected (zombies). 6. **Cross-host replica check** — hash-compare what you pushed to a remote against canonical; flag stale/orphaned/wrong-path copies. Read-only; never touch a live surface without a gate. 7. **Hygiene/security** — secrets in files, key-rotation status, loose permissions, credentials in git history. 8. **Residual-closure** — every review caveat filed "non-blocking / next rung": was it ever dispositioned? Highest-value forgotten items, because they were *known* risks.

9.3 The sweep pattern. For big estates, spawn **parallel read-only sub-agents**, one per slice, same structured brief, one consolidator (§8.2). The brief: *"You are a <AUDIT-TYPE> auditor. Sweep <SCOPE>. Hunt for <patterns>. For each finding return ITEM / SOURCE file:line / AGE / SEVERITY / WHAT-CLOSING-IT-NEEDS. Top-10 ranked. Read-only — do not edit/commit/kill/rotate."* Uniform output → trivial merge. The consolidator dedups, ranks, assigns owners, writes one register.

9.4 The register format — an immediate-actions table (# / item / why-now / owner / effort), then findings by theme (<ID> <SEVERITY> – <line> : <evidence file:line> → <fix> (owner)), then an ownership summary. Disposition codes make triage mechanical: RPT-FIX (reword) · RPT-ADD (new section) · SEAM (build backlog) · SCOPE-OUT (named + rationale) · INV-ONLY · BUILD · OBE (overtaken / already delivered).

9.5 Cadence & triggers — update the resume file + register on every substantive change (continuous, cheap); run a forgotten-items + drift sweep **on return after any gap** (state drifts fastest when attention lapses); a hygiene + anonymization sweep **before any external release**; a drift sweep **after a role/plan change**; and a periodic "what's still open / did we lose anything" pass so armed watches and long jobs don't vanish. Every finding gets an **owner and a disposition** — never bare "blocked" — and the registers themselves get hash-and-errata discipline, because once a register steers the program, corrupting it steers everything.

10. Gates: what requires the human

Enumerate, in writing, exactly what the swarm may **not** do without the Operator: anything that leaves the machine (network egress, sending messages, publishing); integration into the mainline, deployment, going live; changing canonical/persistent state or shared accounts; anything irreversible or high-consequence.

The principle: **gate by reach and consequence, not by the verb**. A shell command that stays inside a sandbox is fine; the *same* command reaching outside needs approval. Gate by *where an effect lands*.

And **approval doesn't generalize**. A yes for one action in one context is not a yes for the next. Durable approval names the action, scope, and (for anything sensitive) an expiry — casual "sounds good" in chat is direction, not a gate-unlock. When in doubt, the agent investigates read-only, proposes, and waits for an explicit yes.

11. Resume protocol: surviving session death

Agent sessions die — context limits, crashes, dropped connections. The swarm must survive with zero lost work: 1. **Write state as it happens**, into the resume and channel files. Never hold important state only in the live conversation. 2. **A fresh session reads the resume file first**, before acting — role, constraints, hazards, pick-up point. 3. **Idempotent, receipted actions** — anything that mutates leaves a receipt (a hash, a log line) so a resumed session can tell what already happened and not redo it. 4. **Verify provenance on resume** — confirm which model/agent you actually are (some harnesses silently fall back to a different model under load); if the running model ≠ the intended one, say so, because it affects who's authoritative and how work is labeled.

If your resume file is good, a total session loss costs nothing but a re-read. That's the bar.

12. Anti-patterns (our scars)

- **In-place editing a cited artifact**. Breaks every reference. Errata only.
- **Trusting "tests pass."** Run them.
- **Same-lineage review theater**. Two siblings agreeing isn't verification.

- **The audited party writing the audit prompt.** Anchors the reviewers — use a neutral primer for high-stakes reviews.
- **Sequential heavy model calls in one foreground task.** Timeout kills the batch. Detach and poll.
- **Silent model fallback.** Verify which model you are on resume.
- **Casual approval treated as a gate-unlock.** Distill to a decision record.
- **Stale role charter on boot.** A fresh Build-Lead reading an old "you are the reviewer" file acts with the wrong authority. Current role at the top of the resume file.
- **Forgotten background work.** Long jobs and standing "held" items vanish if nothing tracks them. Keep an oversight log; sweep periodically.
- **Tight retry loops against a throttling API.** Risks a ban. Back off.
- **Duplicate work.** Two agents running the same sweep is the most wasteful thing in a swarm. One owner per slice.

13. You don't need to know any of this

The part that matters most if you're just starting: **you do not need to have read, memorized, or understood this document to run a swarm like it.** You need to be able to *ask*. Everything here — the channel files, the resume file, the evidence tags, the review scripts, the registers, the drift sweeps — is structure, and structure is exactly what an agent is good at building on request. You hand one capable agent this guide (or just describe what you want), and it stands up the whole apparatus *around itself*, then maintains it.

What you type on day one:

"Read this multi-agent swarm guide. Set us up to work this way. Create the authority-root and channel directories, write my resume file with your current role at the top, drop in the evidence-vocabulary and gate rules as standing constraints, and from now on update the resume file and channel logs on every substantive change. Tell me what you made."

From then on you issue *intents*, in plain language:

"Before you build, run a forgotten-items and drift sweep and tell me what's open." "Get a cross-lineage panel to adversarially review this before I approve it." "You're the reviewer now; the other agent leads. Update the role charter." "Something feels lost from the early plans — fan out read-only agents and trace it."

The agent picks the orchestration, spawns the research agents, manages the cost, writes the registers, and hands you back conclusions and gate-requests. You never wrote a script, learned the file taxonomy, or chose a fan-out shape.

What you keep, and cannot delegate: the **gate** and the **direction**. The agent builds the apparatus and runs it, but *you* decide what gets built, what's acceptable, and what's allowed to happen irreversibly. That judgment is the one thing that stays human — and the one thing you *don't* need technical knowledge for. You need taste, care, and the willingness to say "not yet."

Honest caveats, so this isn't oversold: ask for the discipline explicitly at least once (evidence tags, hash-review, gates) — a fresh agent is capable of all of it but won't always volunteer the rigor unless the conventions are written where it reads them; that's what the resume file is for. Review what it stands up — it'll get the structure right; confirm the *constraints* match what you actually want gated. And the swarm

is only as honest as the conventions you keep: hold the line on the evidence tags and the errata rule, and the rest self-corrects.

The proof is in your hands: this very document was assembled by an agent *inside* a running swarm, on request, from lived practice. That is the whole point. The barrier to entry is not expertise. It is knowing that you can ask, and being willing to hold the gate.

14. Adoption

You don't need all of this on day one. The minimum viable version: 1. Two agents (different vendors), one git repo, one human. 2. Three files: `A_TO_B.md`, `B_TO_A.md`, `CURRENT_HANDOFF.md`. 3. The evidence vocabulary (§5) used in every claim. 4. Immutable artifacts + hash-named review requests (§6). 5. A rule: nothing irreversible without the human (§10).

Add the cross-lineage panel (§7) the first time two agents confidently disagree and you need a tie-breaker. Add parallel sweeps (§8–9) the first time your estate is too big to eyeball. Add the full realization ladder (§5.2) the first time someone mistakes a design doc for a working system — which happens sooner than you think.

Adoption checklist: - [] One authority root; one resume file per agent; role charter kept current. - [] Two directional channel files; entries dated, signed, self-contained. - [] Evidence tags on every claim; realization ladder for "how real." - [] Immutable artifacts + errata; review requests carry bytes+sha256; reviewers hash-first and **run** tests. - [] Cross-lineage panel available for tie-breaks and high-stakes reviews. - [] Standing registers (drift, gaps, mechanism inventory, decision index) with owners. - [] Drift sweeps on return-after-gap, before release, after role change, periodically for background work. - [] Enumerated gates; approval never generalizes; casual "yes" → distilled decision record. - [] Resume-from-file proven by surviving a session kill with zero lost work.

The whole guide collapses to one habit: **make forgetting, drifting, and over-claiming structurally hard — in files — so the agents can be as fast and confident as they are without the swarm losing the thread.**

15. Appendix: copy-paste templates

T1 — Resume file skeleton

```
# <Agent> current handoff
Updated: <date> (update on EVERY substantive change; read BEFORE acting)
**CURRENT ROLE (<operator>, <date>):** <who leads / who reviews / what's frozen>
## Evidence vocabulary
DECISION=operator ruled · VERIFIED=reproduced here · OBSERVED=seen in named source ·
SPECIFIED=frozen contract, unbuilt · CLAIMED=reported not reproduced · UNKNOWN.
## Standing constraints (never violate)
<the hard rules: what needs a gate, what's never touched>
## Known hazards (learned this project)
<the scars — each one bit us once>
## Key paths
<authority root · channels · build workspaces · tooling>
---
# RESUME STATE — <date> (READ FIRST)
```

<the literal pick-up point: what's done, what's next, what's blocked and on whom>

T2 — Channel entry

```
### <date> <time> – <REQUEST | VERDICT | FINDING | HANDOVER> – <one-line>
<self-contained body: readable a week later without surrounding context.
carry evidence tags; carry hashes for anything reviewable.>
– <signed: role, and verified model/agent identity>
```

T3 — Decision record

```
# DECISION – <topic> – <date>
**Ruled by:** <operator>, verified channel <how>. **Status:** immutable.
- **DECISION** <the ruling, verbatim where possible>. Source: <exact ref>.
## Required distinctions / non-voteable floors
<the machine-checkable consequences>
## Open (UNKNOWN)
<what this does NOT settle>
```

T4 — Exact-hash review request

```
### <date> – REVIEW REQUEST: <artifact> (stop on any identity mismatch)
path: <path> bytes: <N> sha256: <full hash>
Review the <design/code> itself, not any prior conclusion. Focus: <load-bearing points>.
Return ACCEPT/REVISE/REJECT with concrete load-bearing findings.
No <live/integration/effect> action is requested.
```

T5 — Review verdict

```
### <date> – <REVIEWER> EXACT REVIEW: <artifact>
Identity VERIFIED: <bytes>/<sha> (exact match, no stop) | model-provenance: <verified id>.
# VERDICT: ACCEPT | REVISE | REJECT – <one-line rationale>
**What carries it:** <the real strengths, specific>.
**Findings:** F1 (load-bearing) ... Fn (minor) – each with file:line + specific fix.
**Tests:** <RAN them: N/N – VERIFIED, not relayed>.
No <live/effect> action taken or authorized.
```

T6 — Cross-lineage panel fan-out (dependency-free)

```
import json, urllib.request, os, sys
key = open(os.path.expanduser("~/gateway_api_key")).read().strip()
doc = open(sys.argv[1]).read()
PRIMER = "CONTEXT: <what the artifact is; what claim it makes; prior review state>."
TASK = ("\\n\\nAdversarially review as a rigorous domain expert. Do NOT rubber-stamp. "
        "Address: (1) over-claim audit vs stated evidence tags; (2) soundness of the "
        "central method; (3) what's MISSING from the frame; (4) the safety/correctness "
        "crux – robust or exploitable; (5) does it hold up as a <artifact type>; "
        "(6) the single weakest claim. Return VERDICT (sound / sound-with-fixes / "
        "needs-rework), numbered findings each with a specific fix, then the single "
        "most important improvement.")
JOBS = [("A", "openai/..."), ("B", "anthropic/..."), ("C", "x-ai/...")] # DIFFERENT lineages
for name, model in JOBS:
    body = {"model": model, "max_tokens": 9000, # big budget: reasoning can eat it all
            "messages": [{"role": "user", "content": PRIMER+"\\n\\n=== ARTIFACT ===\\n"+doc+TASK}]}
    try:
        r = json.load(urllib.request.urlopen(urllib.request.Request(
            "https://<gateway>/v1/chat/completions", data=json.dumps(body).encode(),
```

```

        headers={"Authorization":"Bearer "+key,"Content-Type":"application/json"}), timeout=290)
    msg = r["choices"][0]["message"].get("content","")
    open(f"EXAM_{name}.md","w").write(msg) if msg else print("EMPTY", name,
        r["choices"][0].get("finish_reason"))          # check finish_reason on empties
    except Exception as e: print("ERR", name, str(e)[:120])
# Launch each model DETACHED (nohup) if sequential calls would exceed a foreground timeout.

```

T7 — Model-in-agent-harness (real repo tools via a gateway)

```

export GATEWAY_KEY=$(cat ~/.gateway_api_key)
agent-cli exec \
  -c 'model_providers.gw.base_url="https://<gateway>/v1"' \
  -c 'model_providers.gw.env_key="GATEWAY_KEY"' \
  -c model_provider=gw -m "vendor/model" --sandbox read-only \
  "You are a <role> auditor. Explore this repo (key files: ...). Deliver <output contract> \
  as your final message." > raw.log 2>&1

```

T8 — Evidence + realization quick-reference (pin everywhere)

```

TAGS:  DECISION · VERIFIED · OBSERVED · SPECIFIED · CLAIMED · UNKNOWN
      (over-claiming a tag = a review-blocking defect)
LADDER: L0 theory → L1 spec → L2 isolated-impl → L3 focused-verification
        → L4 integrated → L5 rig-integrated → L6 live
      (no lower rung implies a higher one)
GATE:  nothing that leaves the machine / integrates / goes live / mutates canonical
        state / is irreversible – without the operator's explicit, scoped yes.

```

This guide describes a working method as of its date; the specifics of any one swarm evolve. The conventions — files of record, evidence tags, hash discipline, cross-lineage review, human gates, resume-from-file, and drift sweeps — are the durable part. And the whole of it is available to anyone who can ask an agent to assemble it.